OnionGuard

Local Language Support — Implementation Report

Codebase Investigation

---

1. Executive Summary

This report documents exactly how multi-language support is implemented in the OnionGuard mobile application and backend, based on a direct review of the source code (not documentation or marketing claims). OnionGuard does not use a single, unified localization system. It uses **three separate, independently maintained translation mechanisms**, each covering a different part of the app, with different language coverage and no shared source of truth between them:

Mechanism	Covers	Method	Languages with real content
1. Static UI dictionary	Buttons, labels, nav bar, all app chrome	Hardcoded Dart map, no AI, no network call	6 of 9 (en, tw, dg, ee, ha, fr)
2. Backend voice (gTTS)	Spoken treatment guidance audio	Google gTTS text-to-speech engine	2 of 9 with a correct native voice (en, ha)
3. Live LLM translation	On-screen treatment guide text only	Real-time call to Claude 3 Haiku via OpenRouter	Attempts all 9, quality unverified for 3 low-resource languages

Headline finding: the app's language picker lists 9 languages (English, Twi, Dagbani, Ewe, Hausa, Kusaal, Gurene, Mampruli, Français), matching what is shown in the product presentation deck. In practice, only 6 have genuine, distinct translated UI text; Mampruli's entries are a verbatim copy of Dagbani; Kusaal and Gurene have no dictionary entries at all and silently fall back to Dagbani, then English. Native-accent voice playback works correctly for only 2 languages.

2. System 1 — Static UI Dictionary (Frontend)

File: `onion-guard-mobile/frontend/lib/providers/language_provider.dart`

This is the primary and default translation mechanism for the app. There is no Flutter-native `intl` / `.arb` localization system in use — translations are a hand-written Dart data structure compiled directly into the app.

How it works

a **LanguageProvider** is a `ChangeNotifier` (Flutter's Provider state-management pattern) that holds the current language code as a simple string, e.g. `'en'` or `'tw'`.

b A static map `_translations` holds **129 unique keys** (e.g. `'home'`, `'scan_disease'`, `'welcome'`), each pointing to a nested map of language-code → translated string.

c A second, separate static map `languages` holds just {code: display name} pairs (e.g. `'ku': 'Kusaal'`) and is what populates the dropdown picker in Settings — this is why the picker can show a language that has no actual translated content behind it.

d The lookup function `t(key)` returns the string for the current language; if the current language has no entry for that key, it falls back first to Dagbani (for Kusaal/Gurene specifically, by explicit code comment reasoning that they are related Mabia/Gur languages of northern Ghana), then to English.

e The chosen language is saved locally on the device with `SharedPreferences` under the key `'language'`, so the choice persists across app restarts.

f 13 different page files call `context.watch<LanguageProvider>()` and then `lang.t('key')` wherever UI text is rendered.

Code excerpt — dictionary structure

```
'home': {'en': 'Home', 'tw': 'Fie', 'dg': 'Yili', 'ee': 'A■e', 'ha': 'Gida', 'mp': 'Yili',
'fr': 'Accueil'},
```

Code excerpt — fallback logic (language_provider.dart, lines 31–41)

```
String t(String key) {
  final entry = _translations[key];
  if (entry == null) return key;
  final direct = entry[_languageCode];
  if (direct != null) return direct;
  if (_languageCode == 'ku' || _languageCode == 'gu') {
    final dg = entry['dg'];
    if (dg != null) return dg;
  }
  return entry['en'] ?? key;
}
```

Note: this fallback path only fires for `ku`/`gu`. `mp` *(Mampruli) is not routed through this fallback — it has real dictionary entries for all 129 keys, but every single value is identical to the corresponding Dagbani value, i.e. it was populated by copying Dagbani rather than translating.*

Coverage — System 1 (UI text)

Language	Code	In picker	Distinct translated text (129 keys)
English	`en`	✓ **Yes**	✓ **Yes**
Twi	`tw`	✓ **Yes**	✓ **Yes**
Dagbani	`dg`	✓ **Yes**	✓ **Yes**
Ewe	`ee`	✓ **Yes**	✓ **Yes**
Hausa	`ha`	✓ **Yes**	✓ **Yes**
Français	`fr`	✓ **Yes**	✓ **Yes**
Mampruli	`mp`	✓ **Yes**	*Copy of Dagbani*
Kusaal	`ku`	✓ **Yes**	✗ No
Gurene	`gu`	✓ **Yes**	✗ No

3. System 2 — Treatment Guide Voice (Backend, gTTS)

Files: `backend/treatment-service/app/main.py` and `backend/treatment-service/app/treatment_data.py`

This is a second, completely independent translation surface, only for the audio "Listen" button on the treatment guide screen. It has its own hand-written per-language content and its own language-to-voice mapping, unrelated to System 1.

How it works

a `treatment_data.py` stores a `description` field manually translated into exactly **5 languages**: English, Twi, Dagbani, Ewe, Hausa. The `symptoms`, `treatment_steps`, `prevention`, and `disease_name` fields are English-only in every case.

b The endpoint concatenates disease name + description + treatment steps into one string, then hands it to Google's gTTS (Google Text-to-Speech) library to synthesize an MP3.

c gTTS only has native voice models for a subset of languages. A local `LANG_MAP` decides which voice engine to use per language code — shown below.

Code excerpt — LANG_MAP (main.py, lines 10–16)

```
LANG_MAP = {
  "en": "en",
  "tw": "en",  # gTTS doesn't support Twi natively; use English TTS with Twi text
  "dg": "en",  # Dagbani fallback
  "ee": "en",  # Ewe fallback
  "ha": "ha",  # Hausa is supported by gTTS
}
```

In plain terms: when Twi, Dagbani, or Ewe are selected, the app still plays audio — but it is the **English voice engine reading non-English words phonetically**, which the code itself acknowledges will be mispronounced. Any language code not present in the map (French, Mampruli, Kusaal, Gurene) falls through to `LANG_MAP.get(language, "en")`, i.e. defaults to the English voice reading whatever text it is given — and since those 4 languages also have no entry in `treatment_data.py`, that text is the raw English description itself.

Coverage — System 2 (voice)

Language	Code	Text available (treatment_data.py)	Voice engine used	Correct pronunciation
English	`en`	✓ **Yes**	Native English voice	✓ **Yes**
Hausa	`ha`	✓ **Yes**	Native Hausa voice	✓ **Yes**
Twi	`tw`	✓ **Yes**	English voice reading Twi text	✗ No
Dagbani	`dg`	✓ **Yes**	English voice reading Dagbani text	✗ No
Ewe	`ee`	✓ **Yes**	English voice reading Ewe text	✗ No
Français	`fr`	✗ No	English voice reading English text	✗ No

Mampruli	mp	✗ No	English voice reading English text	✗ No
Kusaal	ku	✗ No	English voice reading English text	✗ No
Gurene	gu	✗ No	English voice reading English text	✗ No

4. System 3 — Live LLM Translation (OpenRouter → Claude 3 Haiku)

Files: `backend/diagnosis-service/app/main.py` (endpoint + LLM call), `frontend/lib/services/openrouter_service.dart` (client), `frontend/lib/pages/treatment_page.dart` (only caller)

This is the only mechanism in the app that calls an external AI language model for translation. It is used in exactly one place — the on-screen (not audio) text of the treatment guide page — and nowhere else in the app.

How it works, step by step

a The treatment guide screen shows English text by default: disease description, symptoms, treatment steps, prevention tips.

b If the user switches to any language other than English, `_onLanguageChanged()` in `treatment_page.dart` bundles all four English text blocks into one labeled string (`DESCRIPTION: ... SYMPTOMS: ... STEPS: ... PREVENTION: ...`) and calls `OpenRouterService.translate(allText, lang)`.

c This hits the backend endpoint `POST /llm/translate`. The backend looks up a human-readable name for the language code in its own `LANG_NAMES` map, builds a prompt ("Translate the following text to {language}. Return ONLY the translated text…"), and sends it to `openrouter.ai/api/v1/chat/completions` using the model configured in `DEFAULT_LLM_MODEL` (currently `anthropic/claude-3-haiku`).

d OpenRouter is a third-party API gateway that forwards the request to Anthropic's Claude and returns Claude's response in a standard chat-completion format. No OpenAI product or key is used anywhere in this codebase.

e The translated block comes back as one string, gets split back into its four sections client-side by matching the original English section headers, and is displayed on screen. If anything fails, the screen silently keeps showing English.

Code excerpt — language names sent to Claude (main.py, lines 187–197)

```
LANG_NAMES = {
  "en": "English", "tw": "Twi (Akan)", "dg": "Dagbani", "ee": "Ewe",
  "ha": "Hausa", "ku": "Kusaal", "gu": "Gurene (Frafra)",
  "mp": "Mampruli", "fr": "French",
}
```

Unlike Systems 1 and 2, this map has a real entry for **all 9 languages**, including Kusaal, Gurene, and Mampruli — so a translation request for any of the 9 will be sent to Claude with a sensible target-language name attached, and Claude will return *something* rather than erroring.

Important caveat — unverified quality for low-resource languages

Kusaal, Gurene, and Mampruli are low-resource languages spoken by relatively small populations in northern Ghana, with very little text available on the public internet compared to English, French, or even Twi/Hausa. Large language models such as Claude have seen comparatively little training data in these languages. The code will not crash or refuse for these languages — it will confidently return a translation — but there is no verification anywhere in the codebase (and none was performed as part of this investigation) that the output is linguistically accurate. This is a meaningful risk for content as safety-relevant as crop disease treatment instructions.

Coverage — System 3 (live LLM translation)

Language	Code	Target name sent to Claude	Expected translation quality
English	`en`	n/a — skipped, shown as-is	N/A (source language)
Twi	`tw`	Twi (Akan)	Likely reasonable (higher-resource)
Dagbani	`dg`	Dagbani	Uncertain (lower-resource)
Ewe	`ee`	Ewe	Likely reasonable (higher-resource)
Hausa	`ha`	Hausa	Likely reasonable (higher-resource)
Français	`fr`	French	Reliable (high-resource)
Mampruli	`mp`	Mampruli	*Unverified / low-resource*
Kusaal	`ku`	Kusaal	*Unverified / low-resource*
Gurene	`gu`	Gurene (Frafra)	*Unverified / low-resource*

5. Master Coverage Matrix (All Three Systems)

This table combines all three systems to show, per language, exactly what a user actually experiences today.

Language	UI text (System 1)	Treatment voice (System 2)	Treatment on-screen text (System 3, live AI)
English	**✓ Yes**	Correct native voice	N/A — source language
Twi	**✓ Yes**	English voice, Twi text (mispronounced)	Live-translated by Claude
Dagbani	**✓ Yes**	English voice, Dagbani text (mispronounced)	Live-translated by Claude
Ewe	**✓ Yes**	English voice, Ewe text (mispronounced)	Live-translated by Claude
Hausa	**✓ Yes**	Correct native voice	Live-translated by Claude
Français	**✓ Yes**	English voice reading English text	Live-translated by Claude
Mampruli	*Copy of Dagbani*	English voice reading English text	Live-translated, unverified quality
Kusaal	✗ No	English voice reading English text	Live-translated, unverified quality
Gurene	✗ No	English voice reading English text	Live-translated, unverified quality

6. Key Findings

a The three translation systems are independently maintained with no shared data source, so their language coverage has drifted apart over time (9 vs. 6 vs. 5 languages with real content, depending on which system is being used).

b The app's language picker (and the product presentation deck) shows 9 languages as if uniformly supported. In reality, Kusaal and Gurene have zero translated content anywhere in the codebase, and Mampruli's UI text is a direct copy of Dagbani, not an independent translation.

c Voice guidance — a headline accessibility feature for low-literacy users — only produces correct native pronunciation for English and Hausa. The other 7 languages either read foreign-language text with an English voice engine (mispronounced) or read English text outright.

d The one place a real AI model is used for translation (System 3, Claude 3 Haiku via OpenRouter) has no quality verification for the three lowest-resource languages (Kusaal, Gurene, Mampruli), which are exactly the languages where an LLM is least likely to perform reliably.